
# Deep Learning

*Tutorial 8A: Object Detection using Faster RCNN (Custom Model)*

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # ***000000494171***

Semester: **4th**

Department of Robotics & Artificial Intelligence

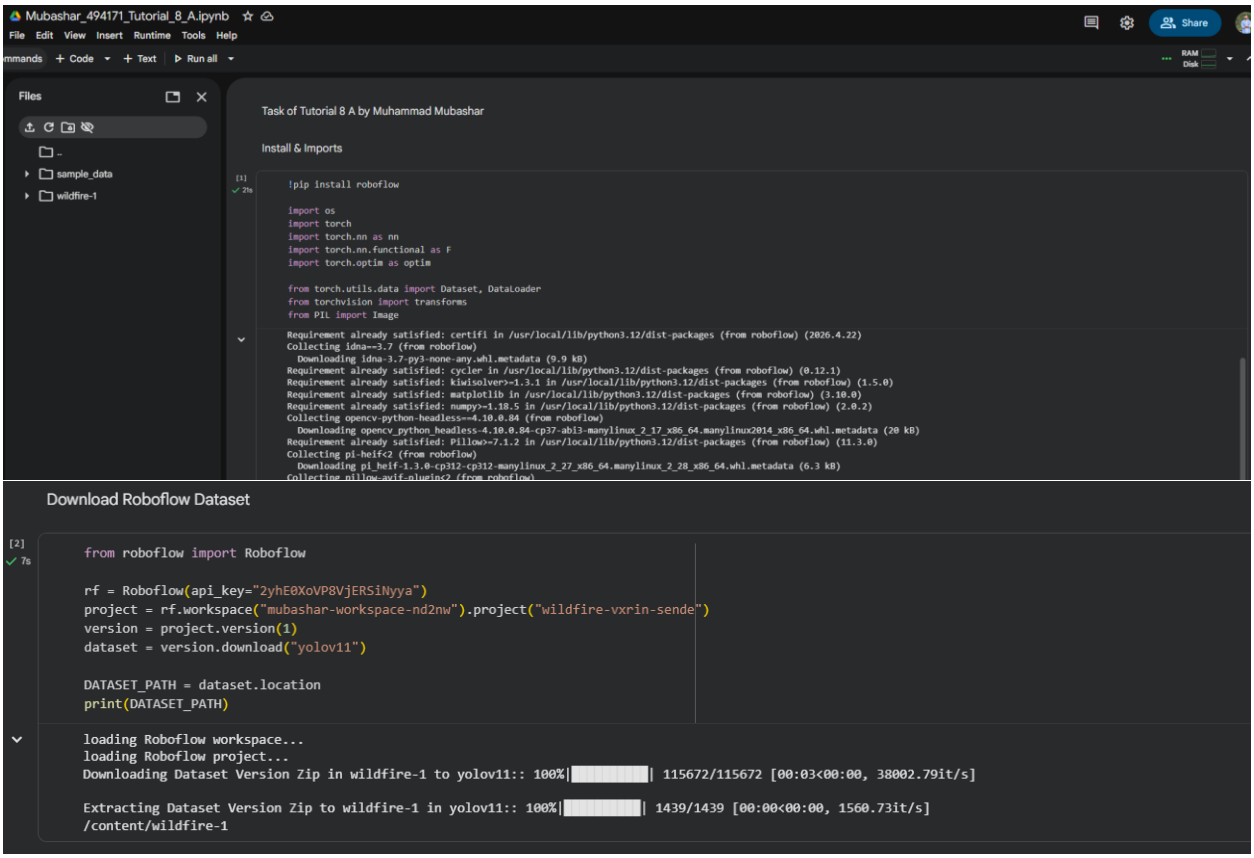
School of Mechanical & Manufacturing Engineering

NUST

## Tasks:

- Change the number of layers
- Take a data if you have and just randomly test it what it shows
- Do a labelling of about 150 images through labelling tools and develop your own model train and test on the data and show the results.

## Implementation of these tasks are in ***PyTorch***:



## Dataset Loader

[3] ✓ 0s

```python
class SimpleDataset(Dataset):
    def __init__(self, root, transform=None):
        self.root = root
        self.transform = transform
        self.images = []

        for file in os.listdir(root):
            if file.endswith(".jpg") or file.endswith(".png"):
                self.images.append(file)

    def __len__(self):
        return len(self.images)

    def __getitem__(self, idx):
        img_path = os.path.join(self.root, self.images[idx])
        image = Image.open(img_path).convert("RGB")

        if self.transform:
            image = self.transform(image)

        # Dummy label (for now)
        label = torch.tensor(0)

        return image, label
```

## DataLoader

[4] ✓ 0s

```python
transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor()
])

train_path = os.path.join(DATASET_PATH, "train/images")

train_dataset = SimpleDataset(train_path, transform)
train_loader = DataLoader(train_dataset, batch_size=2, shuffle=True)

print("Total images:", len(train_dataset))
```

```
Total images: 502
```

## Backbone Network

[5] ✓ 0s

```python
class Backbone(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(128, 256, 3, padding=1), nn.ReLU(),
            nn.Conv2d(256, 512, 3, padding=1), nn.ReLU()
        )

    def forward(self, x):
        return self.conv(x)
```

RPN

[6] ✓ 0s

```python
class RPN(nn.Module):
    def __init__(self, in_channels=512, num_anchors=9):
        super().__init__()
        self.conv = nn.Conv2d(in_channels, 512, 3, padding=1)
        self.cls = nn.Conv2d(512, num_anchors, 1)
        self.reg = nn.Conv2d(512, num_anchors * 4, 1)

    def forward(self, x):
        x = F.relu(self.conv(x))
        return self.cls(x), self.reg(x)
```

ROI Pooling

[7] ✓ 0s

```python
class ROIPool(nn.Module):
    def __init__(self, output_size=(7,7)):
        super().__init__()
        self.pool = nn.AdaptiveMaxPool2d(output_size)

    def forward(self, features):
        return self.pool(features)
```

Detection Head

[8] ✓ 0s

```python
class DetectionHead(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.fc1 = nn.Linear(512*7*7, 1024)
        self.fc2 = nn.Linear(1024, 1024)
        self.cls = nn.Linear(1024, num_classes)
        self.reg = nn.Linear(1024, num_classes * 4)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.cls(x), self.reg(x)
```

Full Faster R-CNN Model

[9] ✓ 0s

```python
class FasterRCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.backbone = Backbone()
        self.rpn = RPN()
        self.roi = ROIPool()
        self.head = DetectionHead(num_classes)

    def forward(self, x):
        features = self.backbone(x)
        rpn_cls, rpn_reg = self.rpn(features)
        pooled = self.roi(features)
        cls, reg = self.head(pooled)
        return cls, reg, rpn_cls, rpn_reg
```

IoU Function

[10] ✓ 0s

```python
def calculate_iou(box1, box2):
    y1, x1, y2, x2 = box1
    Y1, X1, Y2, X2 = box2

    inter_x1 = max(x1, X1)
    inter_y1 = max(y1, Y1)
    inter_x2 = min(x2, X2)
    inter_y2 = min(y2, Y2)

    inter_area = max(0, inter_x2 - inter_x1) * max(0, inter_y2 - inter_y1)

    box1_area = (x2 - x1) * (y2 - y1)
    box2_area = (X2 - X1) * (Y2 - Y1)

    union = box1_area + box2_area - inter_area

    return inter_area / union if union != 0 else 0
```

Training Loop

[11] ✓ 33m

```python
model = FasterRCNN(num_classes=2)
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

for epoch in range(5):
    for images, labels in train_loader:

        optimizer.zero_grad()

        cls, reg, rpn_cls, rpn_reg = model(images)

        loss = criterion(cls, labels)

        loss.backward()
        optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

```
Epoch 1, Loss: 0.0000
Epoch 2, Loss: 0.0000
Epoch 3, Loss: 0.0000
Epoch 4, Loss: 0.0000
Epoch 5, Loss: 0.0000
```